

TIDB × AWS HACKATHON • SÃO PAULO

Ask the Airport

Build a generative AI application in one afternoon. Bring an idea, pick your stack, ship a public repo. The more of the platform you use, the more points you earn — but nothing blocks you from competing.

DATE	BUILD SPRINT	SQUAD SIZE	DEMO	TOTAL POINTS
Wed 02 Sep 2026	2h30	3-4 people	2 min max	100

What you're building

An AI application. That's the brief — the rest is yours.

We're not prescribing the problem. Bring a perspective: an assistant, an analytics tool, a copilot, an agent, something we haven't thought of. The judging rewards clever ideas and genuine business value far more than it rewards checking boxes.

The suggested playing field: airportdb

We've prepared a curated dataset — a realistic airline database of airports, flights, bookings, passengers and weather, filtered to flights touching Brazil across one week in June 2015. Roughly 5,000 flights and 500,000 bookings, small enough to import in minutes.

You don't have to use it. But it's there because it's rich in interesting problems: delays cascade, connections break, weather disrupts, passengers have preferences and histories. Teams that find a real problem in this data and solve it well will have an easier time scoring on creativity and business value than teams starting from a blank sheet.

DATASET **DETAIL**

Window 2015-06-02 to 2015-06-08 (one week)

Scope All flights touching a Brazilian airport, plus a global sample for variety

Size ~5,000 flights · ~500K bookings · weather included

Tables `airline` `airplane` `airport` `booking` `flight` `flightschedule` `passenger`
`weatherdata` and more

Directions worth stealing

- **Natural-language analytics** — a question in, SQL generated, answer out.
- **Disruption copilot** — cross-reference flights and weather to predict which connections break, and explain why in plain language.
- **Travel agent with memory** — learn a passenger's preferences in one conversation, apply them in the next. Rebook them when their flight slips three hours.
- **Semantic concierge** — vector search over routes and notes for "find me something like this".
- **Something else entirely.** Genuinely — the creativity points are real.

How points work

100 points, split between what you built and how you built it. Part A is judged live from your demo. Part B is verified by AI reading your public repo.

TO BE JUDGED, YOU NEED

A **public GitHub repo**, a **SUBMISSION.md** at its root, **TiDB used somewhere** in your application, and a **working demo of 2 minutes or less**. That's the whole floor. Everything else is points, not gates.

Part A — What you built

60 pts

JUDGED LIVE FROM YOUR DEMO

20

It works

The thing runs and does what you say it does. A rough app that works beats a beautiful one that doesn't.

15

Innovation and creativity

An angle we haven't seen four times already today. Surprise us.

15

Business value

Who has this problem, and is this a real answer to it? Name the user.

10

Demo and pitch

Two minutes, one clear story. Show the thing working rather than describing it.

Part B — How you built it

40 pts

VERIFIED BY AI FROM YOUR PUBLIC REPO

10

TiDB Cloud Starter on AWS

Your data lives in a TiDB Cloud cluster in AWS São Paulo (`sa-east-1`). Running TiDB locally instead still keeps you eligible — it just doesn't score here.

8

TiDB vector search

A `VECTOR` column queried with `VEC_COSINE_DISTANCE` , or the `EMBED_TEXT` auto-embedding function. Both count.

8

Amazon Bedrock

Used for text generation, embeddings, or both. Any other LLM API keeps you fully eligible — you just don't score these 8.

8

Deployed on AWS

Lambda with a Function URL, Amplify, Lightsail, EC2, ECS — anything reachable at a URL. Note App Runner is *not* available in `sa-east-1` ; use `us-east-1` if you want it. Running on your laptop is fine; it just scores zero here.

6

Built with Kiro

Commit your `.kiro/` specs and steering files so we can see the spec-driven work, not just the output.

THE HONEST MATH

A team that runs everything locally with no AWS account at all can still score **60 points plus vector search**. A team that uses the whole platform tops out at 100. The gap is real but it is not a wall — a brilliant local build will beat a dull deployed one.

Submitting

Public repo on GitHub, with one file at the root that tells our evaluator what to look for.

Create `SUBMISSION.md` in your repository root and fill it in. Copy this template exactly — the AI evaluator reads it first, then checks your claims against your actual code.

```
# SUBMISSION.md

### Team
Squad name:
Members:

### Pitch
One line on what this does and who it's for.

### What it does
Two or three sentences. What problem, what approach, what the user sees.

### Stack – check what you actually used
- [ ] TiDB Cloud Starter on AWS sa-east-1
- [ ] TiDB vector search (VECTOR column or EMBED_TEXT)
- [ ] Amazon Bedrock
- [ ] Deployed on AWS → live URL:
- [ ] Built with Kiro (.kiro/ committed)

### Where to look
TiDB connection/queries: path/to/file.py
Vector search:           path/to/file.py
Bedrock calls:           path/to/file.py

### Demo
Link to your 2-minute video or live app:
```

ONE RULE ABOUT THE CHECKBOXES

Claims are **verified against your code**, not taken on trust. A ticked box our evaluator can't find in the repository scores zero for that line. Pointing us at the right file in "Where

to look" is the fastest way to get credit for what you actually did.

Commit your secrets to `.env` and your `.env` to `.gitignore` — the repo is public. Include a `.env.example` instead.

Before you arrive

Twenty minutes at home saves you an hour of the sprint. Do these the night before.

TASK	WHY	
GitHub account	You're submitting a public repo	REQUIRED
TiDB Cloud account	tidbcloud.com — free, no card	REQUIRED
Python 3.9+ and pip	Or your language of choice, working	REQUIRED
Clear the Bedrock first-use form	Serverless models are on by default, but Anthropic models need a one-time use-case form per account. If your organisers gave you an account, this is already done	POINTS
Install Kiro and sign in	Credits are issued on the day; the install shouldn't eat your sprint	POINTS
AWS account	For Bedrock and deployment	POINTS

THE SINGLE MOST COMMON WAY TO LOSE AN HOUR

Amazon Bedrock enables serverless models automatically, but Anthropic models still need a **one-time first-use form** the first time an account calls them — until it's submitted you get `AccessDeniedException`. It takes a minute in the Bedrock console, and it's per account, not per person. Clear it before event day. If you're still blocked while building, switch to another LLM API and move on — it costs you 8 points, not your submission.

Quick start

Cluster to querying data in about ten minutes.

1. Create your TiDB cluster

At tidbcloud.com, create a new resource. Choose the **Starter** plan (free), set Cloud Provider to **AWS** and Region to **Sao Paulo (sa-east-1)**. It's active in one to two minutes. Free tier gives you 5 GiB and 50M request units per month — far more than this dataset needs.

Open the cluster, click **Connect**, keep **Public Endpoint**, and copy the host, port (4000), username and database name. The password shows once — save it now or reset it later.

2. Load the dataset

```
curl -O https://hackaton-tidb.s3.sa-east-1.amazonaws.com/dumps/hackathon_airportdb.sql.gz
gunzip hackathon_airportdb.sql.gz

mysql -h <HOST> -P 4000 -u <USERNAME> -p \
  --ssl-mode=VERIFY_IDENTITY --ssl-ca=/etc/ssl/cert.pem \
  < hackathon_airportdb.sql
```

The public endpoint requires TLS — that's what `--ssl-ca` is for. On the venue wifi,

prefer TiDB Cloud's **import from S3** in the console: the cluster pulls the file server-side and never touches your connection.

3. Connect from Python

```
pip install pymysql boto3 python-dotenv
```

```
import os, pymysql
from dotenv import load_dotenv

load_dotenv()

conn = pymysql.connect(
    host=os.environ["TIDB_HOST"],
    port=4000,
    user=os.environ["TIDB_USER"],
    password=os.environ["TIDB_PASSWORD"],
    database="airportdb",
    ssl={"ca": "/etc/ssl/cert.pem"},
)

with conn.cursor() as cur:
    cur.execute("SELECT COUNT(*) FROM flight")
    print(cur.fetchone()[0], "flights loaded")
```

4. Add Bedrock, if you're using it

```
import json, boto3

bedrock = boto3.client("bedrock-runtime", region_name="sa-east-1")

def ask(prompt: str) -> str:
    resp = bedrock.invoke_model(
        modelId="anthropic.claude-3-haiku-20240307-v1:0",
        body=json.dumps({
```



```

        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 500,
        "messages": [{"role": "user", "content": prompt}],
    }},
)
return json.loads(resp["body"].read())["content"][0]["text"]

```

MODELS AVAILABLE IN SA-EAST-1

Amazon Nova is **not** available in São Paulo. For text use `anthropic.claude-3-haiku-20240307-v1:0`, or `mistral.mixtral-8x7b-instruct-v0:1` as an alternative. For embeddings use `amazon.titan-embed-text-v2:0`, which returns 1024 dimensions.

5. Vector search, the short way

TiDB can generate the embeddings for you, so you never call an embedding API from your code:

```

CREATE TABLE flight_notes (
  id BIGINT AUTO_RANDOM PRIMARY KEY,
  flight_id INT,
  note TEXT,
  embedding VECTOR(1024) GENERATED ALWAYS AS (
    EMBED_TEXT("tidbcloud_free/amazon/titan-embed-text-v2", note)
  ) STORED
);

INSERT INTO flight_notes (flight_id, note)
VALUES (1, 'direct flight from Sao Paulo to Frankfurt');

SELECT flight_id, note
FROM flight_notes
ORDER BY VEC_COSINE_DISTANCE(embedding, EMBED_TEXT(
  "tidbcloud_free/amazon/titan-embed-text-v2", 'nonstop to Germany'))
LIMIT 5;

```

Insert text, get vectors. This is the cheapest 8 points on the board.

Spending 2h30

A shape that works. Adjust it, but protect the last fifteen minutes — teams that skip them don't submit.

0:00 – 0:15 **Pick the idea and split up**

Decide the one thing your demo will show. Assign: data, AI, interface, pitch.

0:15 – 0:30 **Cluster up, data in**

One person only. Everyone else starts building against fake data.

0:30 – 1:45 **Build the core loop**

Input to answer, end to end, ugly. Make it work before making it more.

1:45 – 2:05 **Grab the stack points**

Vector column, deploy, commit your `.kiro/` specs. Cheap points, quick wins.

2:05 – 2:20 **Write SUBMISSION.md, push public**

Non-negotiable. Check the repo is actually public.

2:20 – 2:30 **Record the demo, rehearse the line**

Two minutes. Show, don't narrate.

When something breaks

Every failure below has a route around it that keeps you in the competition.

SYMPTOM	FIX, OR ROUTE AROUND IT
AccessDeniedException	The Anthropic first-use form hasn't been submitted for this account. Open any Anthropic model in the Bedrock console and complete it — or swap to another LLM API and carry on. Costs 8 points, nothing else.
ValidationException : model not supported	That model doesn't exist in <code>sa-east-1</code> . Use Claude 3 Haiku, Mixtral, or Titan Embeddings V2.
SSL error connecting to TiDB	The public endpoint requires TLS. Add <code>ssl={ "ca": "/etc/ssl/cert.pem" }</code> in Python, <code>--ssl-ca</code> on the CLI.
Connection times out	Check Network Access on your cluster. Starter allows <code>0.0.0.0/0</code> by default — confirm it wasn't changed.
Dataset import crawling	Venue wifi. Use TiDB Cloud's import-from-S3 in the console instead — the cluster fetches it directly.
No AWS account at all	Build it locally with any LLM API. You keep all 60 Part A points and can still score vector search.
Out of time to deploy	Don't. Run locally, record the demo, submit. Eight points is not worth a missed submission.

IF YOU'RE PROPERLY STUCK

Ask a mentor before you burn fifteen minutes on it. Nobody wins points for debugging alone, and there is always a route that keeps you scoring.

